Experiment - 1

CODE -

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from IPython.display import display

qc = QuantumCircuit(1, 1)

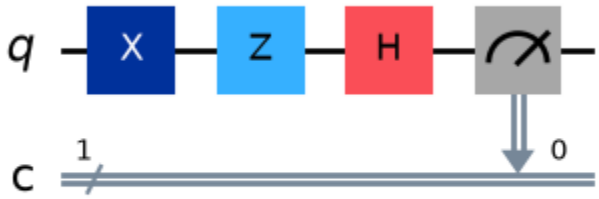
qc.x(0)
qc.z(0)
qc.h(0)
qc.measure(0, 0)

display(qc.draw("mpl"))

simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()

print("Measurement Results:", result.get_counts())
```

OUTPUT -




```
Measurement Results: {'0': 498, '1': 526}
```

Experiment - 2

CODE -

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.quantum_info import Statevector
from qiskit.visualization import (
    plot_histogram,
    plot_state_city,
    plot_state_hinton,
    plot_state_qsphere,
    plot_state_paulivec,
    plot_bloch_multivector
)
from IPython.display import display

qc = QuantumCircuit(2, 2)

qc.x(1)
qc.h(0)
qc.cx(0, 1)

display(qc.draw("mpl"))

state = Statevector.from_instruction(qc)

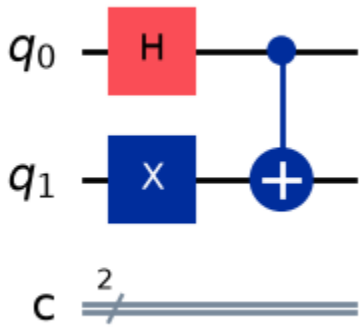
display(plot_state_city(state))
display(plot_state_hinton(state))
display(plot_state_qsphere(state))
display(plot_state_paulivec(state))
display(plot_bloch_multivector(state))

measured_qc = qc.copy()
measured_qc.measure([0, 1], [0, 1])

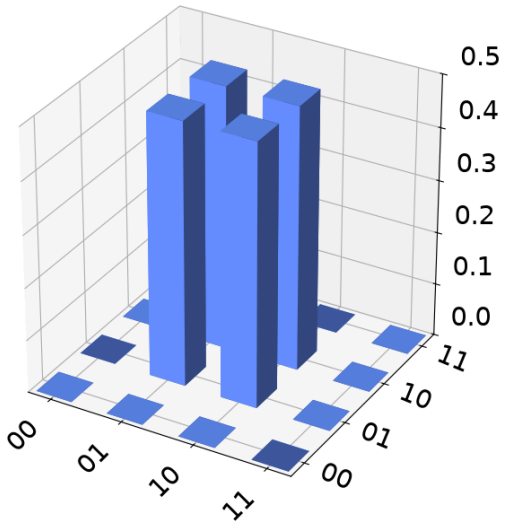
simulator = AerSimulator()
result = simulator.run(measured_qc, shots=1024).result()
counts = result.get_counts()

display(plot_histogram(counts))
print("Measurement Results:", counts)
```

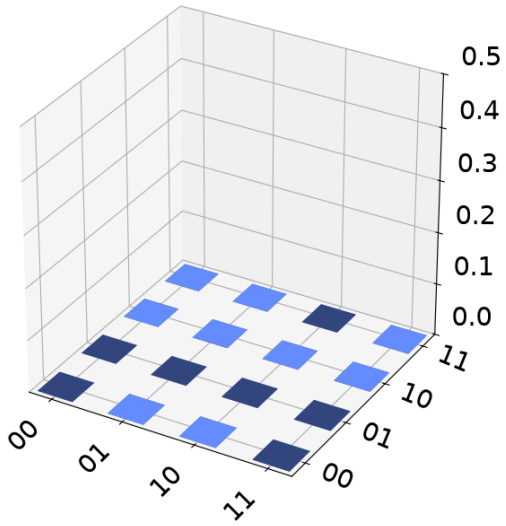
OUTPUT -



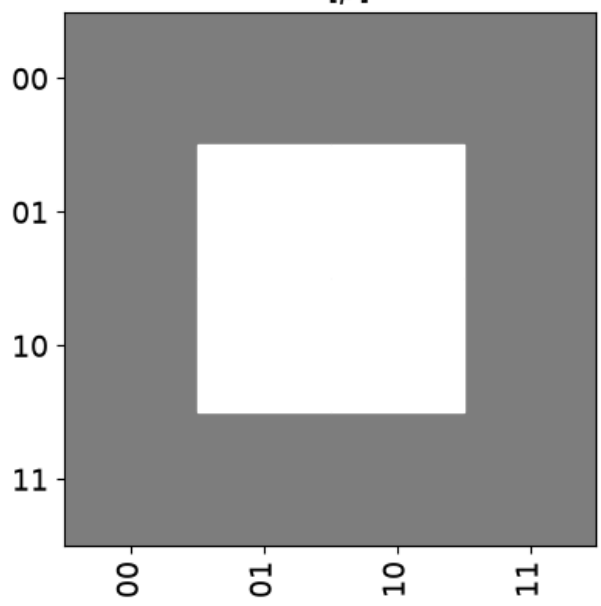
Real Amplitude (ρ)



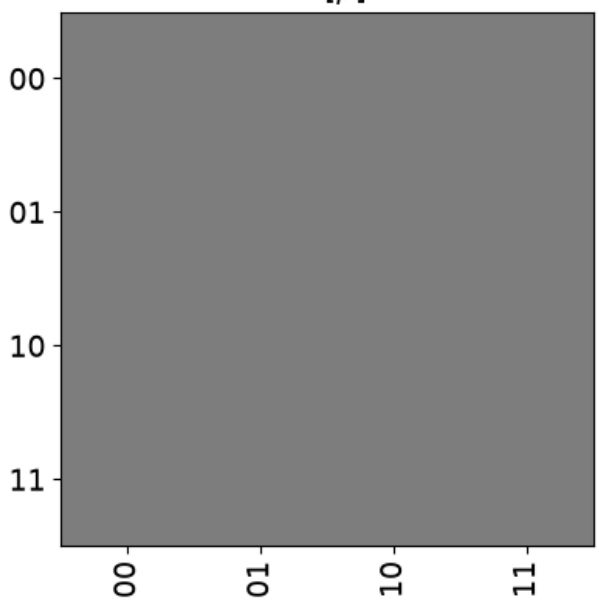
Imaginary Amplitude (ρ)

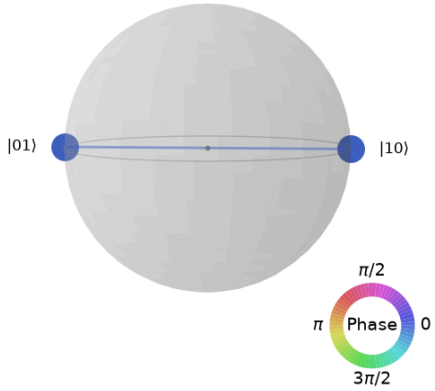


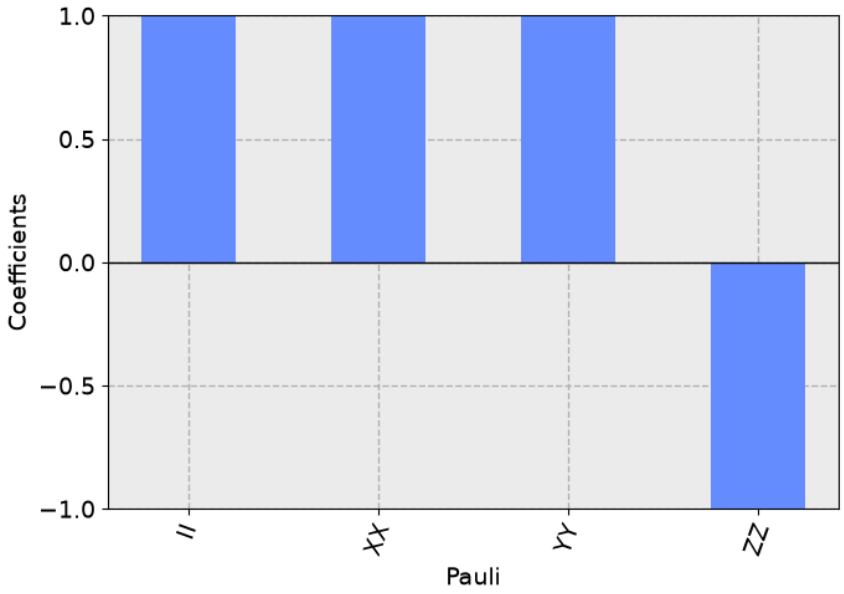
Re[ρ]

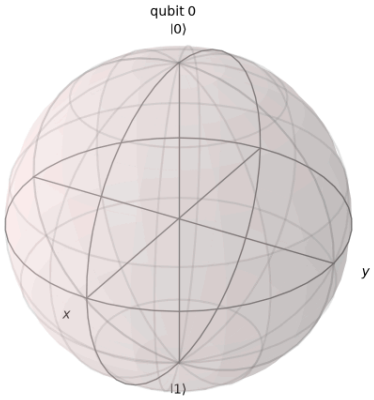


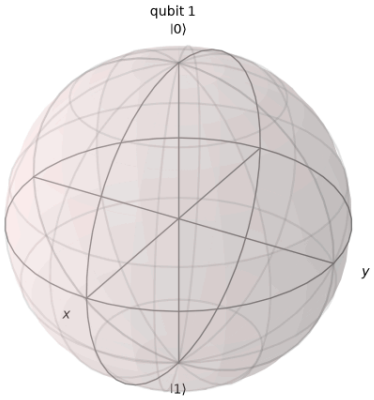
Im[ρ]

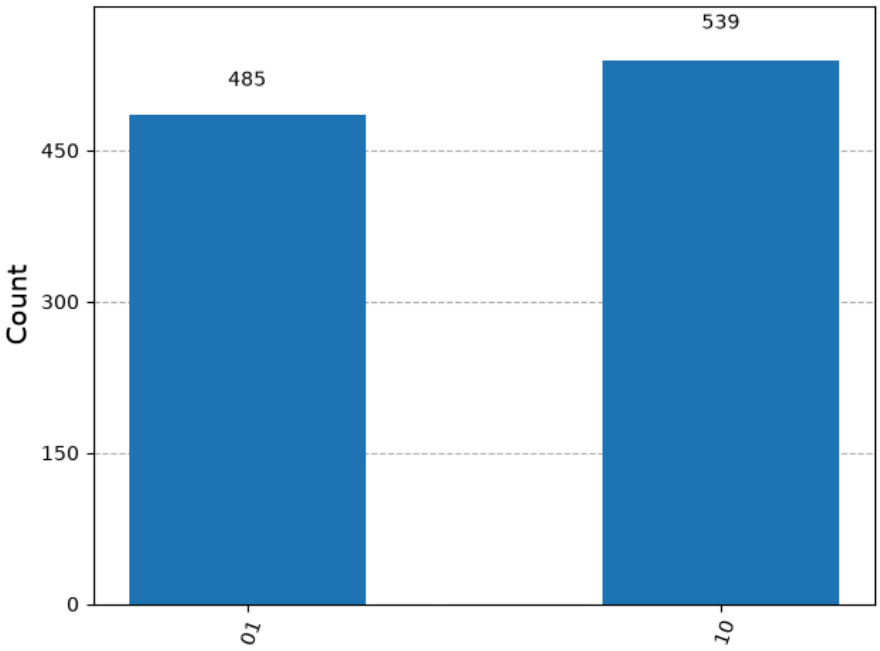












Measurement Results: {'10': 539, '01': 485}

Experiment - 3

CODE -

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.quantum_info import Statevector
from qiskit.visualization import (
    plot_histogram,
    plot_state_city,
    plot_state_hinton,
    plot_state_qsphere,
    plot_state_paulivec,
    plot_bloch_multivector
)
from IPython.display import display

qc = QuantumCircuit(2, 2)

qc.h(0)
qc.cx(0, 1)

display(qc.draw("mpl"))

state = Statevector.from_instruction(qc)

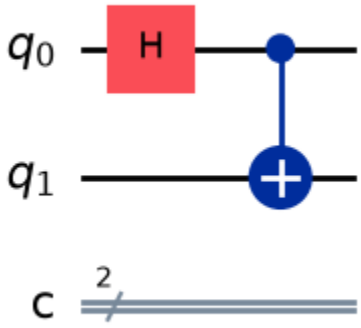
display(plot_state_city(state))
display(plot_state_hinton(state))
display(plot_state_qsphere(state))
display(plot_state_paulivec(state))
display(plot_bloch_multivector(state))

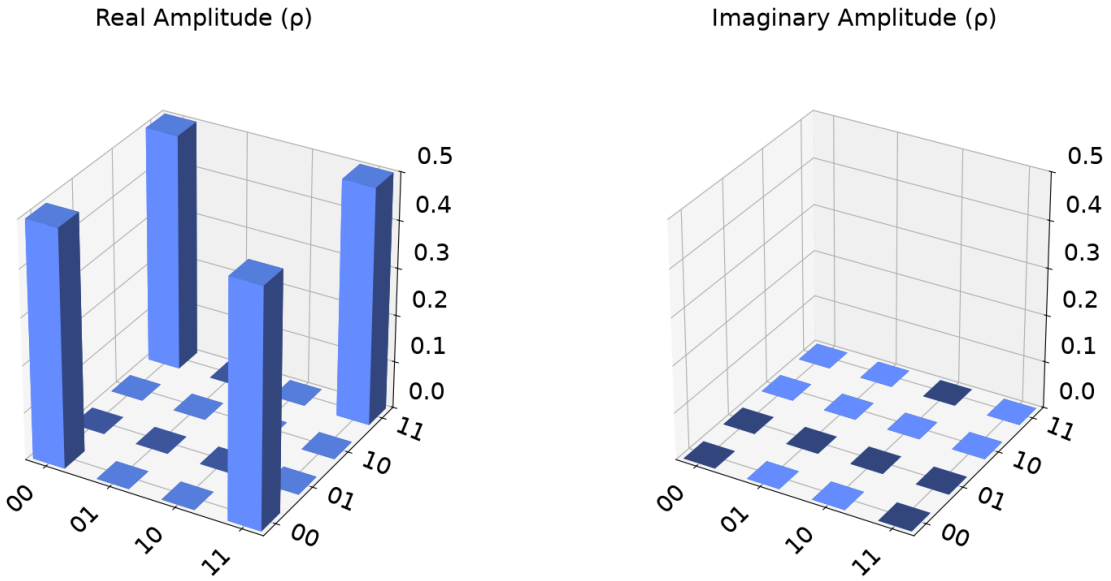
qc.measure([0, 1], [0, 1])

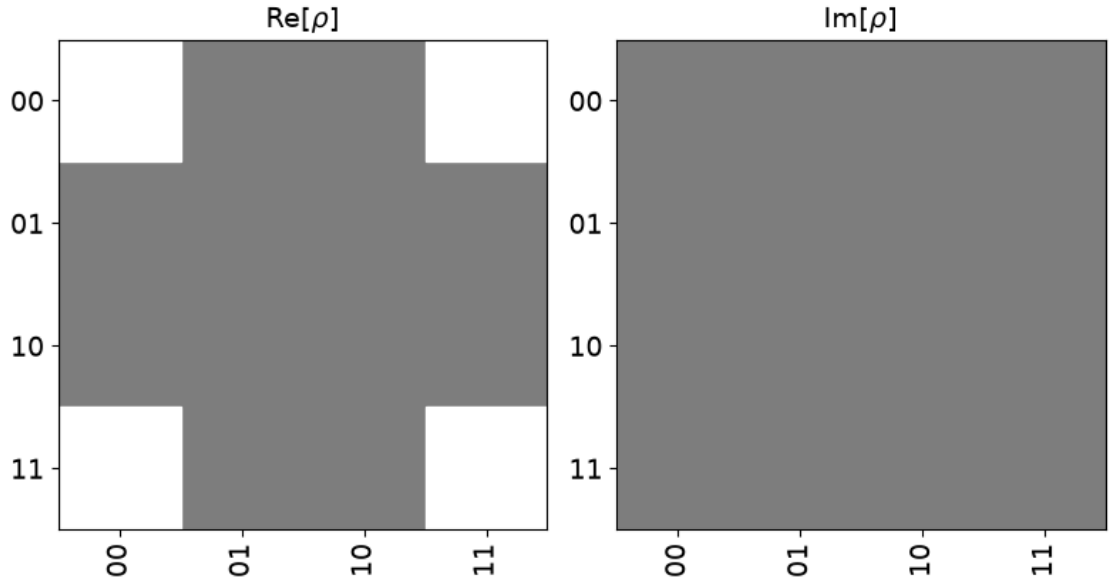
simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()
counts = result.get_counts()

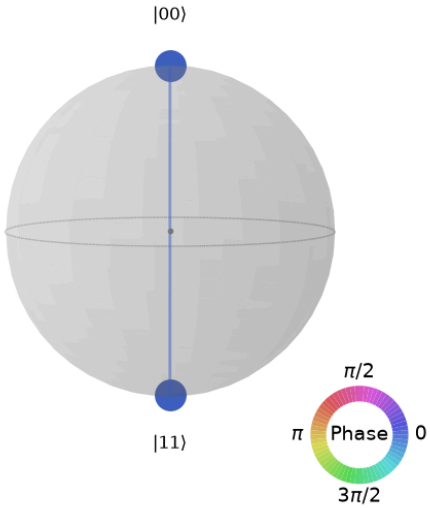
display(plot_histogram(counts))
print("Measurement Results:", counts)
```

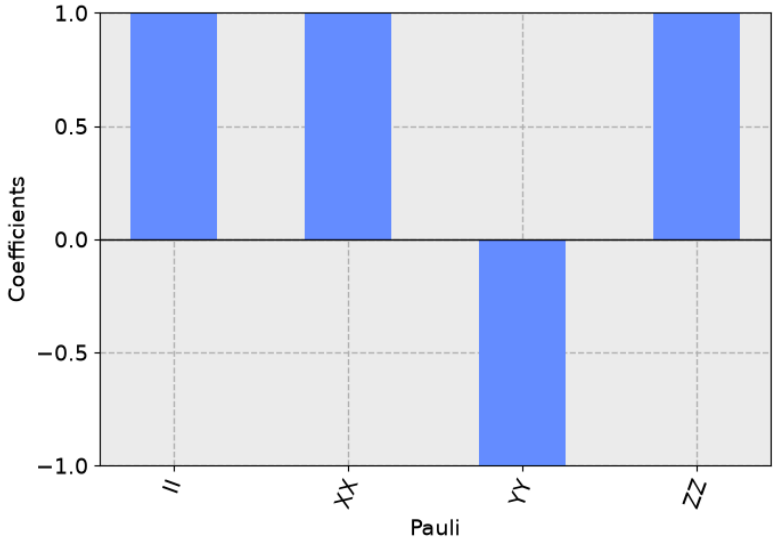
OUTPUT -

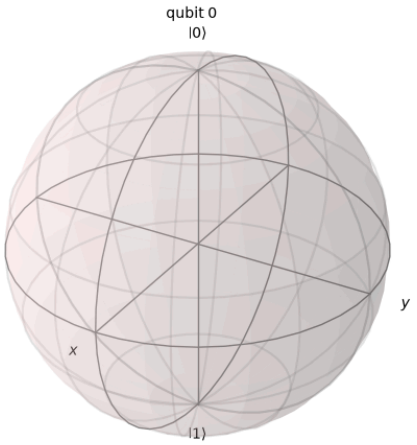

q_0
H
q_1
c
2

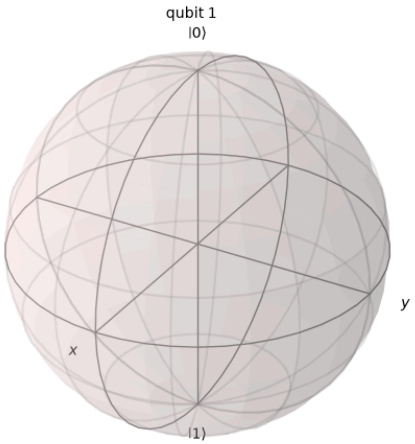

Real Amplitude (ρ)
Imaginary Amplitude (ρ)
0.5
0.4
0.3
0.2
0.1
0.0
00
01
10
11

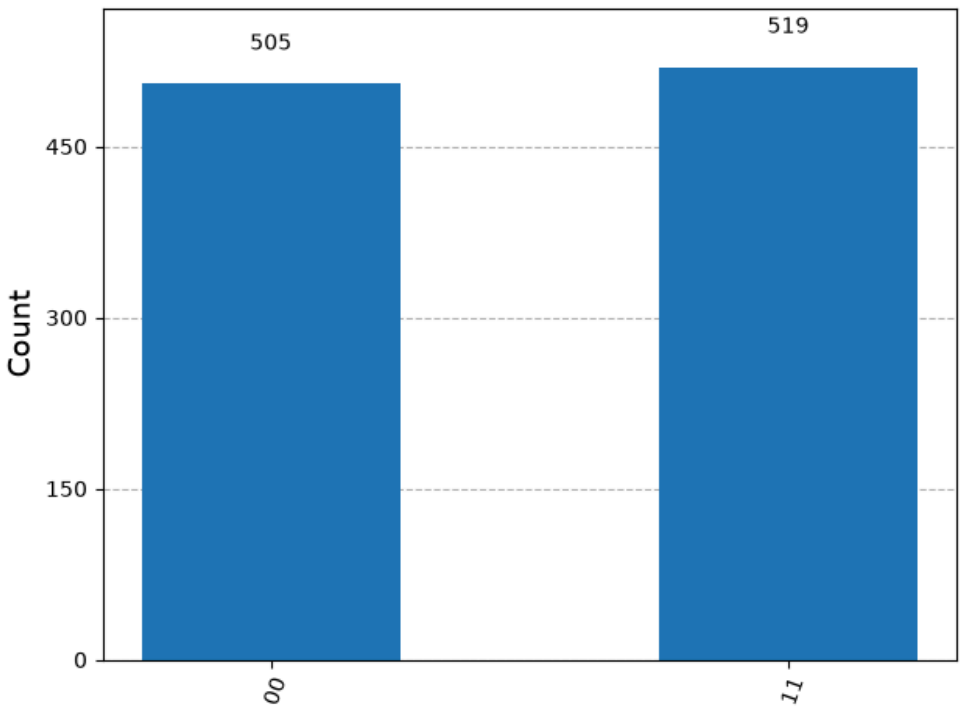

Re[ρ]
Im[ρ]
00
01
10
11











Measurement Results: {'00': 505, '11': 519}

Experiment - 4

CODE -

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from IPython.display import display

messages = ["00", "01", "10", "11"]
simulator = AerSimulator()

for message in messages:
    qc = QuantumCircuit(2, 2)

    qc.h(0)
    qc.cx(0, 1)

    if message == "01":
        qc.z(0)
    elif message == "10":
        qc.x(0)
    elif message == "11":
        qc.z(0)
        qc.x(0)

    qc.cx(0, 1)
    qc.h(0)
    qc.measure([0, 1], [0, 1])

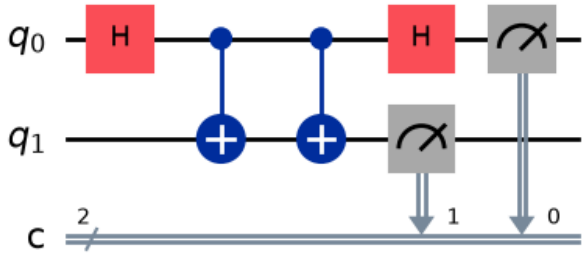
    print(f"Message: {message}")
    display(qc.draw("mpl"))

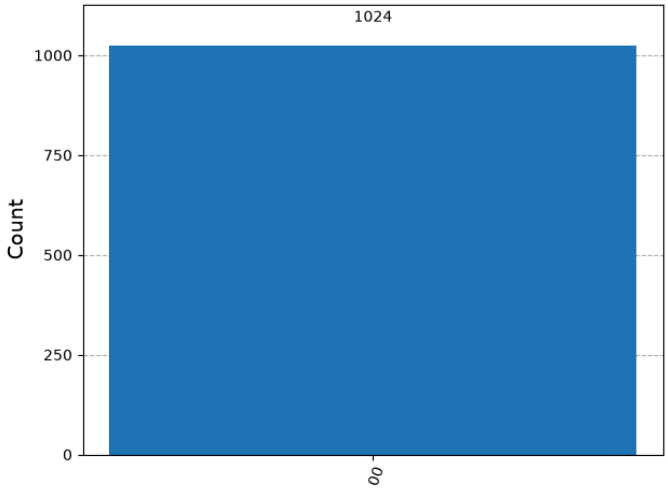
    result = simulator.run(qc, shots=1024).result()
    counts = result.get_counts()

    display(plot_histogram(counts))
    print("Measurement Results:", counts)
```

OUTPUT -

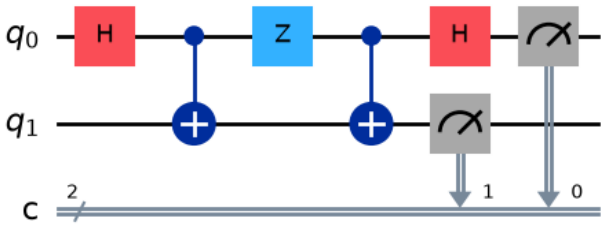
Message: 00

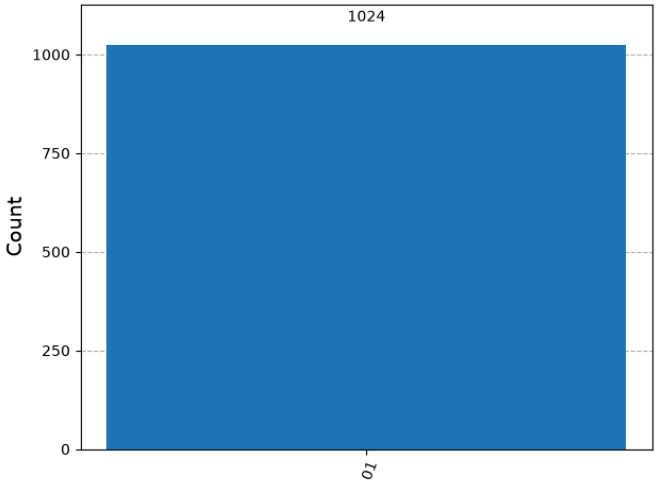




Measurement Results: {'00': 1024}

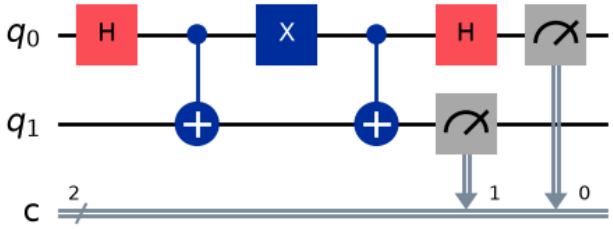
Message: 01

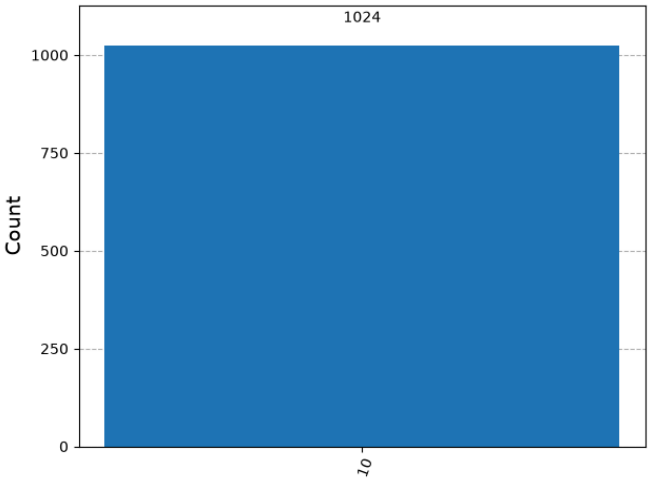




Measurement Results: {'01': 1024}

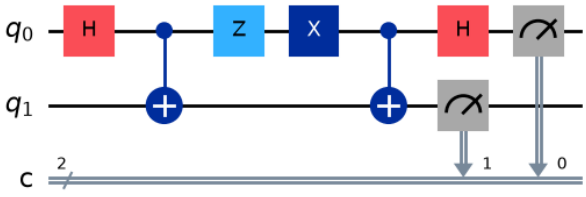
Message: 10

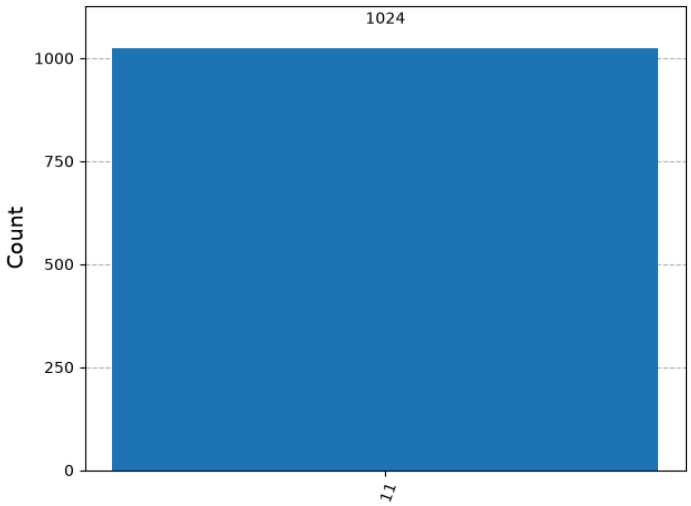




Measurement Results: {'10': 1024}

Message: 11





Measurement Results: {'11': 1024}

Experiment - 5

CODE -

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from IPython.display import display

states = ["01", "10", "11"]
simulator = AerSimulator()

for state in states:
    qc = QuantumCircuit(2, 2)

    if state[0] == "1":
        qc.x(1)
    if state[1] == "1":
        qc.x(0)

    qc.cx(0, 1)
    qc.cx(1, 0)
    qc.cx(0, 1)

    qc.measure([0, 1], [0, 1])

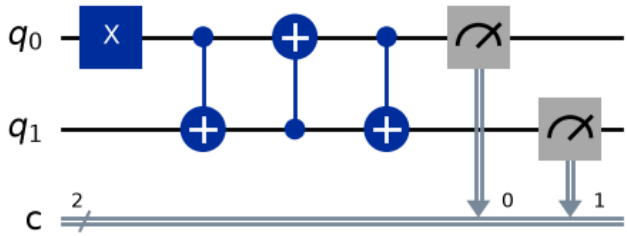
    print(f"Input State: |{state}>")
    display(qc.draw("mpl"))

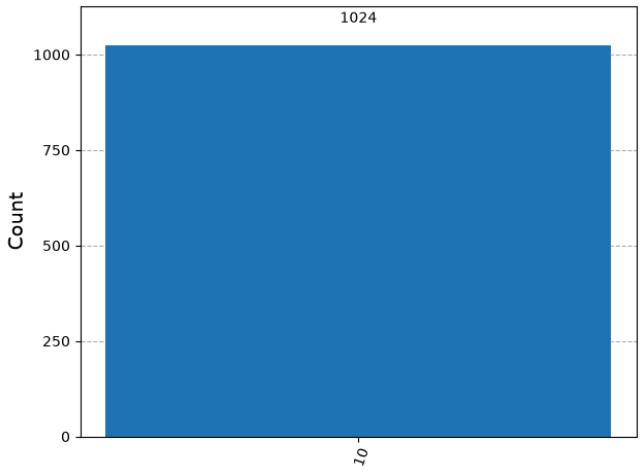
    result = simulator.run(qc, shots=1024).result()
    counts = result.get_counts()

    display(plot_histogram(counts))
    print("Measurement Results:", counts)
```

OUTPUT -

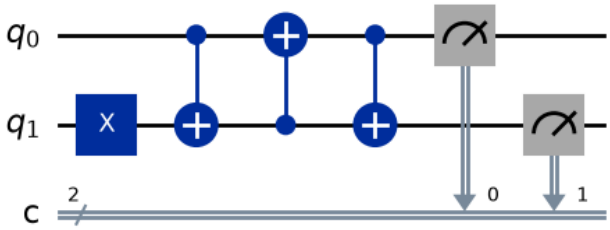
Input State: |01>

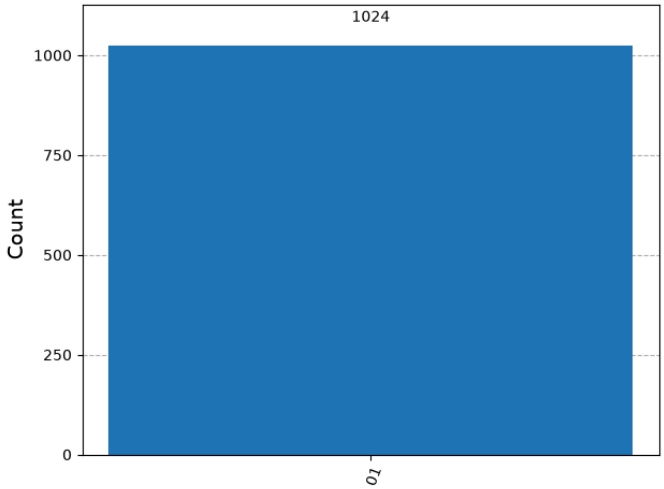




Measurement Results: {'10': 1024}

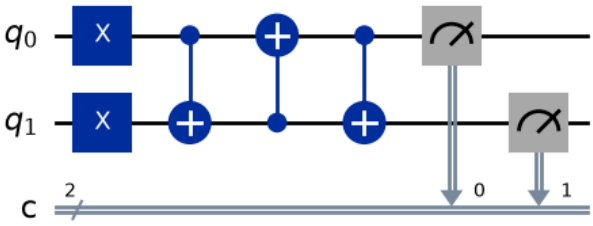
Input State: |10>

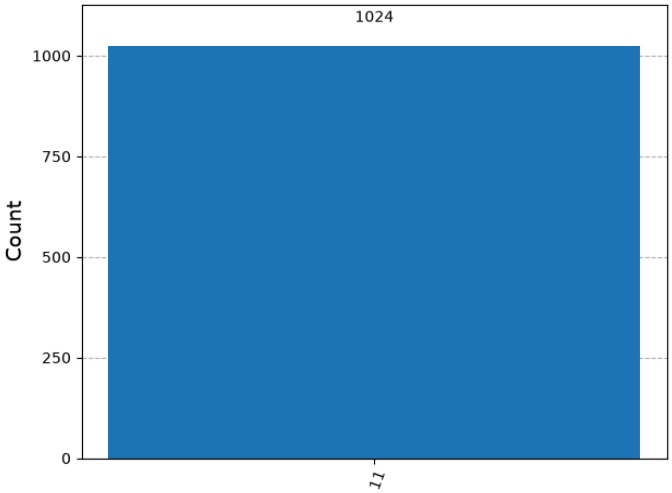




Measurement Results: {'01': 1024}

Input State: |11>





Measurement Results: {'11': 1024}

Experiment - 6

CODE -

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from IPython.display import display

simulator = AerSimulator()
n = 2

for oracle_type in ["Constant", "Balanced"]:
    qc = QuantumCircuit(n + 1, n)

    qc.x(n)
    qc.h(range(n + 1))

    if oracle_type == "Balanced":
        qc.cx(0, n)

    qc.h(range(n))
    qc.measure(range(n), range(n))

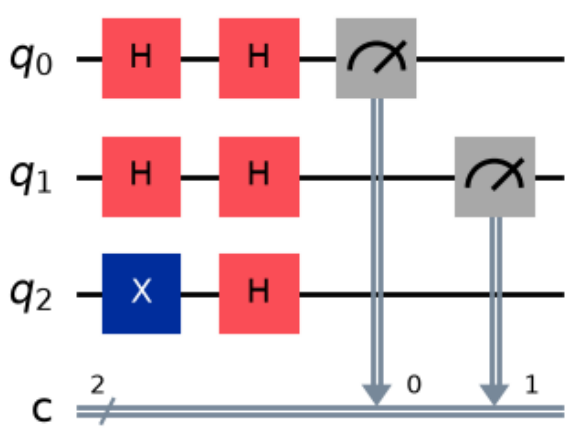
    print(f"{oracle_type} Oracle")
    display(qc.draw("mpl"))

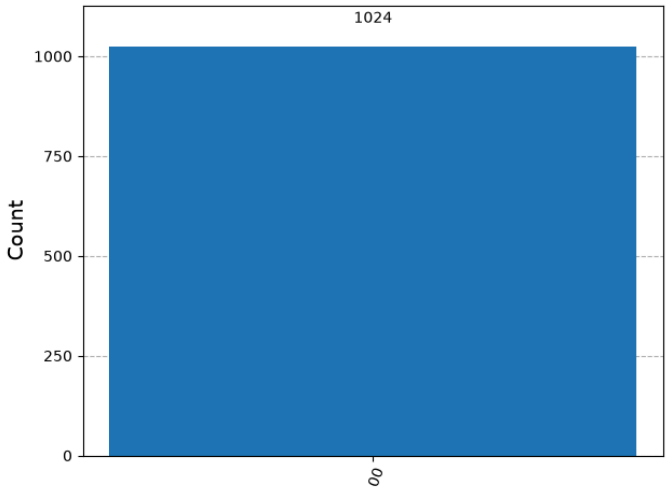
    result = simulator.run(qc, shots=1024).result()
    counts = result.get_counts()

    display(plot_histogram(counts))
    print("Measurement Results:", counts)
```

OUTPUT -

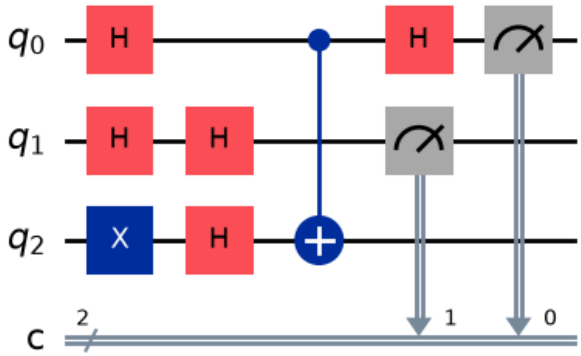
Constant Oracle

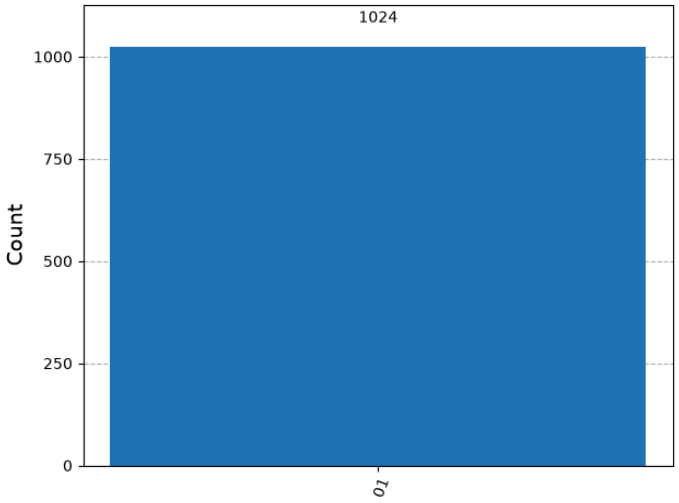




Measurement Results: {'00': 1024}

Balanced Oracle





Measurement Results: {'01': 1024}

Experiment - 7

CODE -

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from IPython.display import display

qc = QuantumCircuit(2, 2)

qc.h([0, 1])

qc.cz(0, 1)

qc.h([0, 1])
qc.x([0, 1])
qc.h(1)
qc.cx(0, 1)
qc.h(1)
qc.x([0, 1])
qc.h([0, 1])

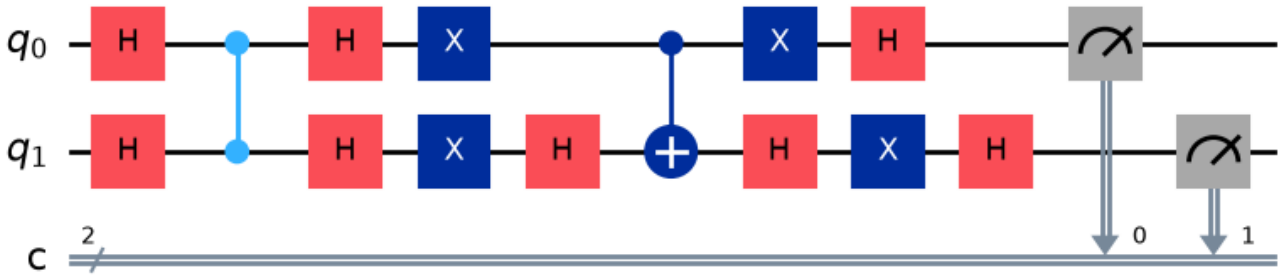
qc.measure([0, 1], [0, 1])

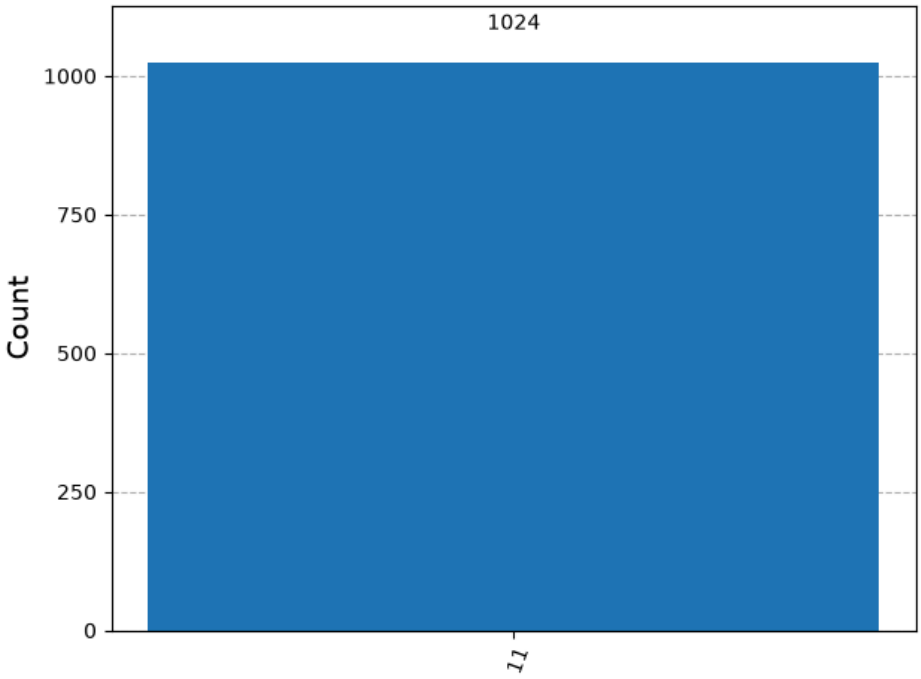
display(qc.draw("mpl"))

simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()
counts = result.get_counts()

display(plot_histogram(counts))
print("Measurement Results:", counts)
```

OUTPUT -





Measurement Results: {'11': 1024}

Experiment - 8

CODE -

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_histogram
from IPython.display import display
from math import gcd
from numpy import pi

def qft(qc, n):
    for i in range(n):
        qc.h(i)
        for j in range(i + 1, n):
            qc.cp(pi / (2 ** (j - i)), j, i)

    for i in range(n // 2):
        qc.swap(i, n - i - 1)

n = 3
qc = QuantumCircuit(n, n)

qc.x(0)
qft(qc, n)

display(qc.draw("mpl"))

state = Statevector.from_instruction(qc)
print("QFT Statevector:")
print(state)

qc.measure(range(n), range(n))

simulator = AerSimulator()
result = simulator.run(qc, shots=1024).result()
counts = result.get_counts()

display(plot_histogram(counts))
print("Measurement Results:", counts)

N = 15
a = 2

print("\nShor's Algorithm")
print("N =", N)
print("a =", a)
print("gcd(a, N) =", gcd(a, N))

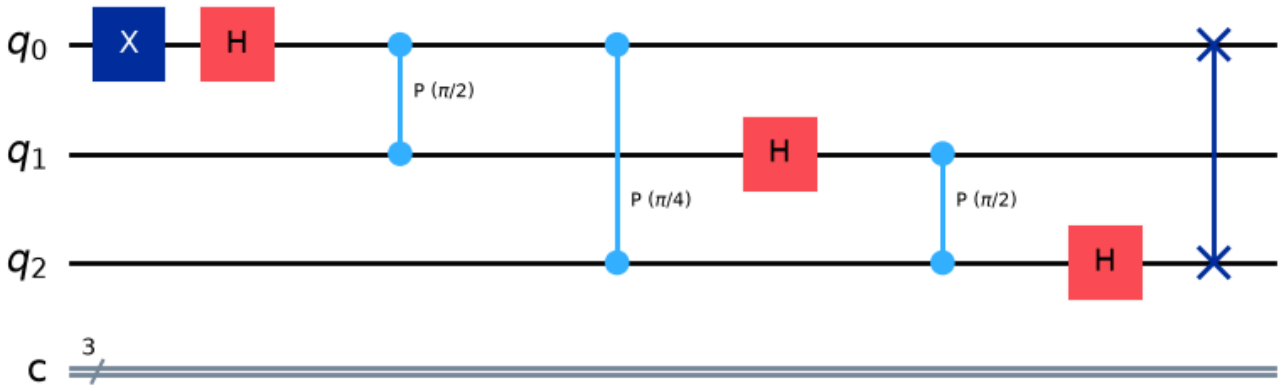
r = 1
```

```
while pow(a, r, N) != 1:
    r += 1

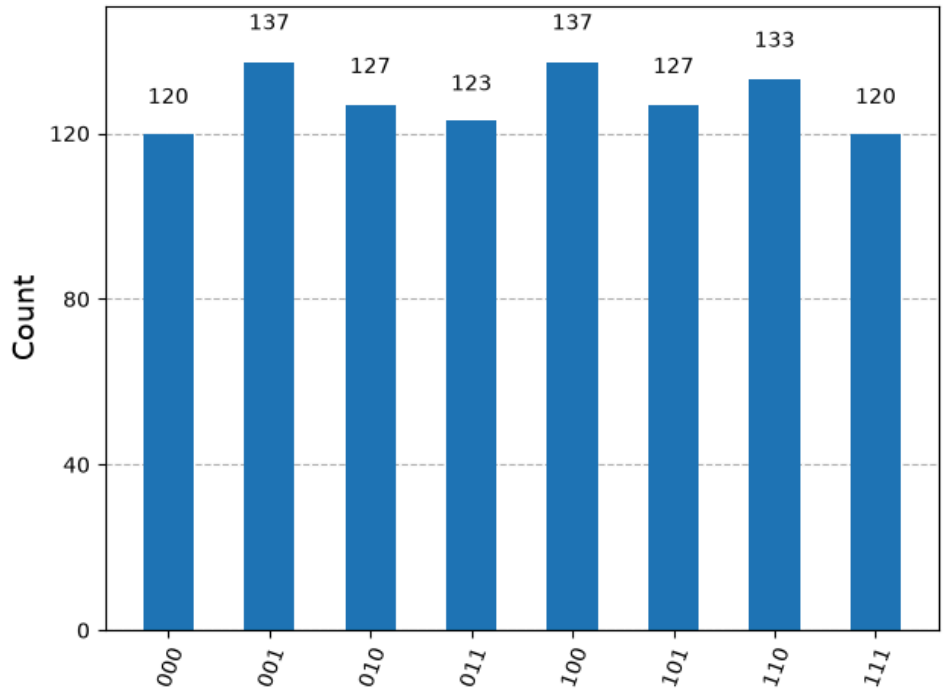
print("Order r =", r)

if r % 2 == 0 and pow(a, r // 2, N) != N - 1:
    factor1 = gcd(pow(a, r // 2, N) - 1, N)
    factor2 = gcd(pow(a, r // 2, N) + 1, N)
    print("Factors:", factor1, "and", factor2)
```

OUTPUT -



QFT Statevector:
Statevector([0.35355339+0.j, 0.35355339+0.j, 0.35355339+0.j,
0.35355339+0.j, -0.35355339+0.j, -0.35355339+0.j,
-0.35355339+0.j, -0.35355339+0.j],
dims=(2, 2, 2))



Measurement Results: {'101': 127, '100': 137, '110': 133, '001': 137, '111': 120, '011': 123, '010': 127, '000': 120}

Shor's Algorithm
N = 15
a = 2
gcd(a, N) = 1
Order r = 4
Factors: 3 and 5